

Real Course 1 对象和类

对象和类

在开始机器人编程实践之前，我更希望大家能够了解一些常用的编程技巧。（这真的很重要！）

面向过程 VS. 面向对象

面向过程就像将编程当成是做一件事，要按**步骤**完成，每一步就是一个过程。比如作菜，先放油，接着是放菜进去炒，然后放水，最后菜就做好了。这里面放油，炒，放水就是三个步骤。

面向对象就是将编程当成是一个事物，对外界来说，事物是直接使用的，**不用去管他内部的情况**。而编程就是设置事物能够做什么事。其实有点像是将面向过程给放到事物内部了。仍然举作菜为例，其实**面向过程**就好像你是个厨师，要自己炒菜，所以要讲究步骤，而**面向对象**就好像你是个食客，你只要通知厨师作菜，即发一个消息就可以了，至于厨师怎样作菜，是不用知道的。

在学习面向对象之前，至少需要对面向过程编程有一点了解（至少语法得都认识吧）

何时创建面向对象的程序

简单的编程任务可以用简单的函数来实现。然而，随着任务的规模和复杂度的增加，函数会变得更加复杂和难以管理。

当函数变得太大时，您可以将它们分成较小的函数，并将数据从一个函数传递到另一个函数。然而，随着函数数量的增加，设计和管理传递给函数的数据也变得困难和容易出错。这种情况下，您可以考虑面向对象的程序设计。

对某些问题来说，从对象的角度来考虑会更简单、更自然。将问题陈述中的名词视为要定义的对象，动词视为要执行的操作。

接下来，将介绍对象和类的概念。在此，我先断言你已经掌握了Java的基本语法

概念: 对象和类

- **对象**：对象是类的一个实例，有状态和行为。
- **类**：类是一个模板，它描述一类对象的行为和状态。

对象与类的差别

对象是具体的，有该对象特定**属性值和方法**的。

类是抽象的（相对于对象而言），只定义一类对象**可以有的属性和方法**，不定义**每个对象特定**的属性值和方法。

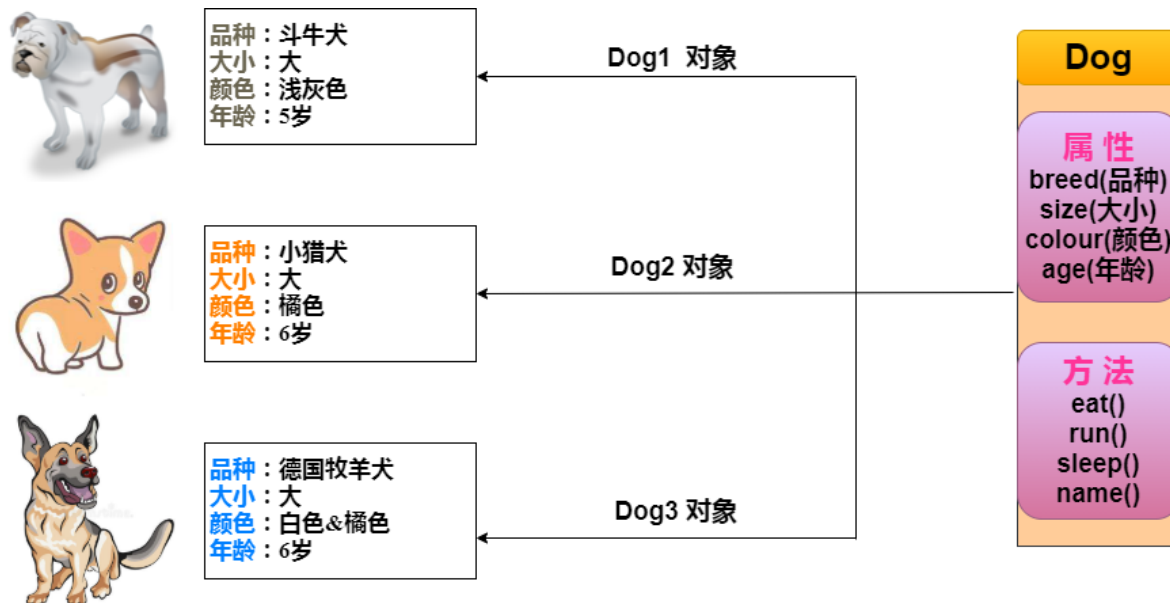
对象可以被想象成一个具体的物体，类可以被想象成将一类物体抽象提炼出一个概念。比如你笔盒里有一支短铅笔、另有一支钝铅笔，他们都从属于铅笔这一抽象概念。铅笔这个概念描述了两支铅笔有石墨和粘土构成的铅芯和木制的外壳、可以在纸上写字，但是铅笔这个**概念**是没有这些**物质性**的特征的。（你不能手握一个虚无缥缈的概念书写，对吧？）这就是类与对象的差别。在这个例子中，铅笔是类，一支短铅笔和另一支钝铅笔是两个对象。

对象和类的理解

对象是类的一个实例。我们也可以用真实世界的**类**与**对象**来理解编程中的对应概念。

拿一条狗来举例，**一条狗**是一个**对象**，它的状态有：名字、品种、颜色，行为有：叫、摇尾巴和跑；**狗**类则是一个**类**，代表着一系列有着相似属性、方法的物种。

我们借此理解软件中的概念，**软件对象也有状态和行为**。**软件对象的状态就是属性**，**行为通过方法体现**。



你可能会理解为：类是一个集合，而对象是一个元素。我不反对，但是不要完全依赖集合的概念去理解类，后续的有些概念会有出入！

Java 中的类

类可以看成是创建 Java 对象的模板。下面是一个简单的狗类定义。

```
public class Dog {  
    String breed;  
    int size;  
    String colour;  
    int age;  
  
    void eat() {  
    }  
  
    void run() {  
    }  
  
    void sleep(){  
    }  
  
    void name(){  
    }  
}
```

一个类可以包含以下类型变量：

- **局部变量**：在方法、构造方法或者语句块中定义的变量被称为局部变量。变量声明和初始化都是在方法中，方法结束后，变量就会自动销毁。
- **成员变量**：成员变量是定义在类中，方法体之外的变量。这种变量在创建对象的时候实例化。成员变量可以被类中方法、构造方法和特定类的语句块访问。
- **类变量**：类变量也声明在类中，方法体之外，但必须声明为 static 类型。

一个类可以拥有多个方法，在上面的例子中：eat()、run()、sleep() 和 name() 都是 Dog 类的方法。

接下来，我们来详细介绍类的具体定义方式

声明一个类

```
public class Dog {  
}
```

这句代码表示声明了一个类，名字为Dog，可全局访问¹，其内部的方法在大括号里面编写。

定义一个方法

```
public class Dog {  
    public void say(String word){  
        System.out.println(word);  
    }  
}
```

这句代码往狗类里面加入了一个say的方法，可以让小狗说话，我们来仔细看看这个方法

- **访问修饰符**（例中为public）：表示该方法允许被使用的范围（访问权限）
- **返回类型**（例中为void）：表示该方法返回值的类型（如int, String, void.....），具体是什么取决于方法内return语句返回的数据类型，若没有return语句，则返回类型为void。
- **方法名**（例中为say）：方法的名字
- **参数**（例中为word）：在调用这个方法时，将会接受一个**参数**，被存储为String变量word，在方法的具体实现中，可以直接把word作为变量使用²。

成员变量

我们往狗类中加入一个成员变量：年龄

```
public class Dog {  
    int age;  
    public void say(String word){  
        System.out.println(word);  
        System.out.println("I'm " + this.age + " years old")  
    }  
    public void grow(){  
        this.age += 1;  
    }  
}
```

这个代码展示了如何定义一个成员变量，并在方法里使用他。

我们需要注意，如果我们要使用一个定义在**类的主体**中成员变量（我的意思是，不是定义在方法内部），我们需要加上this关键字。

```
public class Dog {
    private int age;
    public void say(String word){
        System.out.println(word);
    }
    public void setAge(int age){
        this.age = age;
    }
    public int getAge(){
        return this.age;
    }
}
```

实例化一个类（创建对象）

```
public static void main(String[] args) {
    // 创建对象
    Dog myDog = new Dog();
    myDog.say("Hello!");
    myDog.age = 3;
}
```

实例化对象的语法有点类似变量，但是又多了些什么

- 类型（例中为Dog）：对象的类型（对象对应的类）
- 对象名字（例中为myDog）：对象的名字
- new关键字
- Dog()：创建一个Dog类型的对象，注意后面的括号！后面你会知道他有什么用。

之后，我们可以用 `对象.方法()` 来使用其方法。

不仅如此，我们也可以属于 `对象.成员变量` 来访问其成员变量（甚至修改他）

构造方法

每个类都有构造方法。如果没有显式地为类定义构造方法，Java 编译器将会为该提供一个默认构造方法。**构造方法的名称必须与类同名**，一个类可以有多个构造方法³。构造方法在一个对象被使用**new**关键字被创建时立刻被调用。

```
public class Dog {
    int age;
    String name;
    public void Dog(String name){
        this.name = name;
        this.age = 1;
    }
    public void say(String word){
        System.out.println(word);
    }
}
```

```

        System.out.println("I'm " + this.age + " years old")
    }

    public void grow(){
        this.age += 1;
    }
}

```

我们定义了一个构造方法，可以在创建对象时就为小狗赋予名字，并设置年龄为1岁

加入构造方法后，我们创建对象的代码就需要相应地做出改变。

```

public static void main(String[] args) {
    // 创建对象
    Dog myDog = new Dog("Lucky");
    myDog.say("Hello!");
    myDog.age = 3;
}

```

关注：传入的Lucky是构造方法的参数！也就是说，在new的这一行里，构造方法被调用了。

成员变量的保护

如果，我们写的代码就像这个狗类直接发布给大众，绝对有可能会对我们的类进行这样的篡改。

```

public static void main(String[] args) {
    Dog myDog = new Dog("Lucky");
    myDog.age = 99999;
}

```

我们当然不希望这种事情发生，所以需要age变量进行加密

我们可以把成员变量的访问权限设置为private，防止从其他文件的代码直接访问到变量

我们可以为他们设置专门的get方法，让变量只**可读**。

```

public class Dog {
    private int age;
    private String name;
    public void Dog(String name){
        this.name = name;
        this.age = 1;
    }

    public void say(String word){
        System.out.println(word);
        System.out.println("I'm " + this.age + " years old")
    }

    public void grow(){
        this.age += 1;
    }

    public int getAge(){
        return this.age;
    }
}

```

```
    }

    public int getName(){
        return this.name;
    }
}
```

这样，我们只能调用 `mydog.getAge()` 来读取变量，而无法修改他。

当然，`mydog.age` 也不起作用了，因为访问权限已经设置为了 `private`。

这个代码习惯叫做**封装**，只能通过写好的定义 `get/set` 方法来访问变量，来保护一些变量防止被恶意修改。

```
public class Dog {
    private String name;

    public void setName(String name){
        this.name = name;
    }

    public int getName(){
        return this.name;
    }
}
```

以上代码针对 `name` 这一变量展示了完整的封装。

*补充：访问修饰符

Java中，可以使用访问控制符来保护对类、变量、方法和构造方法的访问。Java 支持 4 种不同的访问权限。

- **default** (即默认，什么也不写)：在同一包内可见，不使用任何修饰符。使用对象：类、接口、变量、方法。
- **private**：在同一类内可见。使用对象：变量、方法。 **注意：不能修饰类（外部类）**
- **public**：对所有类可见。使用对象：类、接口、变量、方法
- **protected**：对同一包内的类和所有子类可见。使用对象：变量、方法。 **注意：不能修饰类（外部类）。**

我们可以通过以下表来说明访问权限：

修饰符	当前类	同一包内	子孙类(同一包)	子孙类(不同包)	其他包
<code>public</code>	Y	Y	Y	Y	Y
<code>protected</code>	Y	Y	Y	Y/N ⁴	N
<code>default</code>	Y	Y	Y	N	N
<code>private</code>	Y	N	N	N	N

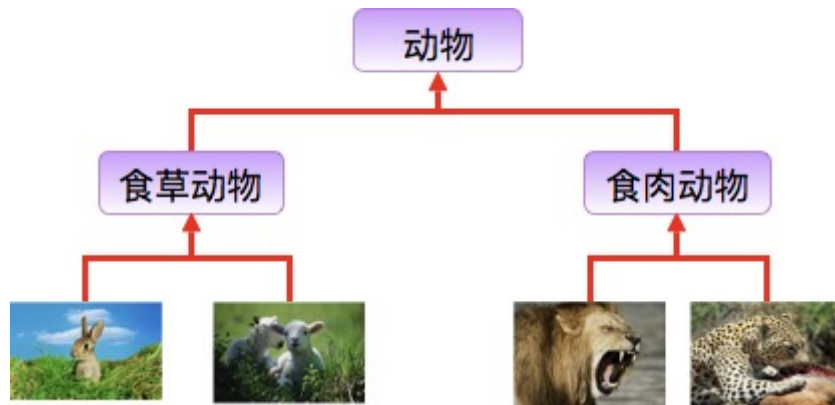
类之间的关系

理论上，目前为止你就可以直接进行机器人编写，以下关于继承，重写等的操作在机器人实际编写中使用不多。不过我仍然建议你了解一下以下内容。

继承

1.概念

继承就是子类继承父类的特征和行为，使得子类对象（实例）具有父类的实例域和方法，或子类从父类继承方法，使得子类具有父类相同的行为。以下是便于理解的一个例子：



在实际编程中，我们可以把几个功能有重复，或者有相关性的类抽象出一个**父类**，并让**子类**分别继承**父类**，可以提高维护性，代码也更加简洁，提高代码的复用性。

2.Java代码

```
class 父类 {  
}  
  
class 子类 extends 父类 {  
}
```

当子类继承这个父类之后，就具有父类当中的属性和方法，子类就不会存在重复的代码。

例如：有一个汽车类和巴士类

```
public class Car{  
    private String name;  
    public Car(String name){  
        this.name = name;  
    }  
    public void drive(){  
        System.out.println(name+"开车了");  
    }  
}
```

```

public class Bus{
    private String name;
    public Bus(String name){
        this.name = name;
    }
    public void drive(){
        System.out.println(name+"开车了");
    }
    public void pay(){
        System.out.println("你坐上了公交车，付钱！");
    }
}

```

那么我们可以提炼出一个父类Vehicle类

```

public class Vehicle{
    private String name;
    public Vehicle(String name){
        this.name = name;
    }
    public void drive(){
        System.out.println(name+"开车了");
    }
}

```

那么Car类与Bus类就可以这样写

```

public class Car extends Vehicle{
    public Car(String name){
        super(name);
    }
}

```

```

public class Bus extends Vehicle{
    public Bus(String name){
        super(name);
    }
    public void pay(){
        System.out.println("你坐上了公交车，付钱！");
    }
}

```

注意：super关键字指向父类，相当于父类的this

3. 方法的重写(Override)

重写是在继承关系的基础上，重写父类已有的方法，写成一个子类的新方法。在调用子类的该方法时，使用子类的方法而不是父类的方法。

继续以上面的车子的代码为例，以下是重写的一个例子，你可以体会一下其作用：

```

public class Bus extends Vehicle{
    public Bus(String name){

```



```

        super(name);
    }

    @Override
    public void drive(){
        // 重写了父类的drive()方法
        System.out.println(name+"开车了，开的很慢");
    }

    public void pay(){
        System.out.println("你坐上了公交车，付钱！");
    }
}

```

这时在使用Bus类的drive方法，就会输出 "开车了，开的很慢"

抽象与实现

1.抽象类

在面向对象的概念中，所有的对象都是通过类来描绘的，但是反过来，并不是所有的类都是用来描绘对象的，如果一个类中没有包含足够的信息来描绘一个具体的对象，这样的类就是抽象类。抽象类除了不能实例化对象之外，类的其它功能依然存在，成员变量、成员方法和构造方法的访问方式和普通类一样。由于抽象类不能实例化对象，所以抽象类必须被继承，才能被使用。也是因为这个原因，通常在设计阶段决定要不要设计抽象类。

省流：**抽象类**可以不写方法的具体**实现**，只规范他有什么方法，需要在后续代码中**实现或重写**

在Java中，用abstract关键字来声明一个抽象类/函数

```

public abstract class Vehicle{
    private String name;
    public Vehicle(String name){
        this.name = name;
    }
    // 这是一个抽象方法，在父类中，没有具体实现，需要每个子类各自实现这个方法
    public abstract void drive(){}
}

```

2.重写

在实现中也非常简单，使用@Override关键词来重写方法就可以输入具体实现

```

public class Bus extends Vehicle{
    public Bus(String name){
        super(name);
    }

    @Override
    public void drive(){
        // 实现了父类的drive()方法
        System.out.println(name+"开车了，开的很慢");
    }

    public void pay(){
        System.out.println("你坐上了公交车，付钱！");
    }
}

```

```
}
```

3.接口

如果一个类里面全是抽象的方法，那就是一个**接口**，我们这样来定义一个接口

```
[可见度] interface 接口名称 [extends 其他的接口名] {  
    // 声明变量  
    // 抽象方法  
}
```

- 接口不能有具体的方法**实现**，也就是说，方法的里面必须是空的！
- 接口不能用来**实例化**！必须有其他类来**继承** (implements)
- 接口里面除了有default标明的方法，必须在继承时全部实现！
- 接口可以多继承（同一个类可以继承多个接口）

我们可以把Vehicle抽象成一个接口，大概就像这样：

```
public interface Vehicle{  
    private String name;  
    public void drive();  
    public default void pay();// 不是所有的载具需要pay，所以若不实现就默认为空  
}
```

那么Bus的代码就会变成这样：

```
public class Bus implements Vehicle{  
    public Bus(String name){  
        super(name);  
    }  
  
    @Override  
    public void drive(){  
        // 实现了父类的drive()方法  
        System.out.println(name+"开车了，开的很慢");  
    }  
  
    @Override  
    public void pay(){  
        System.out.println("你坐上了公交车，付钱！");  
    }  
}
```

那么抽象有什么优点呢，可以方便后续代码的调用和维护，比如**多态**。

多态

情景：我们现在作为一个人，需要去乘坐这些载具了，我们可以自己开车，也可以做公交车（当然，还有很多出行方式）

那么，我们想要去使用载具的行为就像这样

```
public class Person{  
    Bus bus = new Bus("bus");  
}
```

```

Car car = new Car("car");
public void go(String choice){
    switch(choice){
        case "Car":
            car.drive();
        case "Bus":
            bus.drive();
    }
}
public static void main(String[] args){
    go("Bus");
}
}

```

你不难发现，这个代码如同答辩，因为每多增加一种出行方式，我们就要在switch例多写一个判断。没有可维护性。

但是如果我们让两个类继承自Vehicle类的话，我们就可以这样写。

```

public class Person{
    public static void go(Vehicle vehicle){
        vehicle.drive();
    }
}
public static void main(String[] args){
    Bus bus = new Bus("bus");
    go(bus);
}
}

```

Bus作为一个Vehicle的子类，可以作为这里参数的Vehicle类型直接传入，并调用drive()方法。

当然，如果有其他载具，只需要让他继承Vehicle类，不需要改变这里人的逻辑，增强了可延展性。

此外，如果不同载具的drive逻辑不同，只需要在继承时重写该方法即可，通过调用vehicle.drive()可以做到不同载具的不同逻辑，此处就不赘述了。

练习

以下是一个网上摘抄下来的简单的类，我已经好心地保留了注释，看看这个类有些什么功能

Employee.java

```

import java.io.*;

public class Employee {
    private String name;
    private int age;
    private String designation;
    private double salary;

    // Employee 类的构造器
    public Employee(String name) {
        this.name = name;
    }
}

```

```

    }

    // 设置 age 的值
    public void setAge(int age) {
        this.age = age;
    }

    // 获取 age 的值
    public int getAge() {
        return age;
    }

    // 设置 designation 的值
    public void setDesignation(String designation) {
        this.designation = designation;
    }

    // 获取 designation 的值
    public String getDesignation() {
        return designation;
    }

    // 设置 salary 的值
    public void setSalary(double salary) {
        this.salary = salary;
    }

    // 获取 salary 的值
    public double getSalary() {
        return salary;
    }

    // 打印信息
    public void printEmployee() {
        System.out.println(this);
    }

    // 重写 toString 方法
    @Override
    public String toString() {
        return "名字: " + name + "\n" +
            "年龄: " + age + "\n" +
            "职位: " + designation + "\n" +
            "薪水: " + salary;
    }
}

```

猜测一下在另一个文件的main方法里输入这些内容，会输出什么

EmployeeTest.java

```

import java.io.*;

public class EmployeeTest {
    public static void main(String[] args) {

```

```
// 使用构造器创建两个对象
Employee empOne = new Employee("RUNOOB1");
Employee empTwo = new Employee("RUNOOB2");

// 调用这两个对象的成员方法
empOne.setAge(26);
empOne.setDesignation("高级程序员");
empOne.setSalary(1000);
empOne.printEmployee();

empTwo.setAge(21);
empTwo.setDesignation("菜鸟程序员");
empTwo.setSalary(500);
empTwo.printEmployee();
}
}
```

-
1. 关于访问修饰符的介绍，见 *补充：访问修饰符 [↗](#)
 2. 注意：参数变量有作用域，只能在方法内部使用 [↗](#)
 3. 构造方法的名字必须和类名保持一致，然而可以利用参数的不同来实现多个构造方法，这涉及方法的重载(Overload) [↗](#)
 4. 这取决于基类是否在同一个包 [↗](#)